

Value-Passing to Pure CCS Transpiler

Languages for concurrency and distribution project

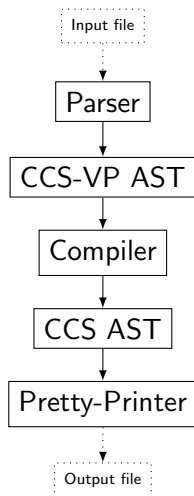
Jacopo Angeli 2130757

Università degli studi di Padova

March 2026

- 1 Overview
- 2 Input Language
- 3 Encoding Rules
- 4 AST Types
- 5 Compiler
- 6 Program
- 7 Demo & Limitations

Overview



- Implements the encoding $\llbracket \cdot \rrbracket : \text{CCS-VP} \rightarrow \text{CCS}$ from lecture
- Input: value-passing CCS with **interval-typed** parameters
- Parser: built with FsYacc/FsLex
- Compiler: structural recursion on the CCS-VP AST
- Output: grounded Pure CCS

Input Language

Declaration:

```
P = proc;  
Q(x:[L,H], y:[L,H]) = proc;
```

Process constructors:

'a(e).P	output
a(x:[L,H]).P	input
tau.P	silent action
P + Q	choice
P Q	parallel
P \ {a:[L,H]}	restriction
P [a/b:[L,H]]	renaming (a to b)
if b then P else Q	conditional
K(e1,...,en)	constant call
nil	nil

Example

```
B = in(x:[0,1]).'out(x).B;  
B' = 'in(0).B';
```

```
Q(x:[0,2]) =  
  if x != 0 then tau.Q(x-1)  
  else 'done(0).nil;
```

Result

```
B = in_0.'out_0.B + in_1.'out_1.  
B
```

```
B' = 'in_0.B'
```

```
Q_0 = 'done_0.0 // 0 = nil  
Q_1 = tau.Q_0  
Q_2 = tau.Q_1
```

Encoding Rules

$$\begin{aligned}
 [a(x:[L, H]). P] &= \sum_{m=L}^H a_m \cdot [P\{^m/x\}] \\
 [\bar{a}(e). P] &= \bar{a}_m \cdot [P] \quad (e \downarrow m) \\
 [\tau. P] &= \tau \cdot [P] \\
 [P + Q] &= [P] + [Q] \\
 [P \mid Q] &= [P] \mid [Q] \\
 [0] &= 0 \\
 [P \setminus \{c:[L, H]\}] &= [P] \setminus \{c_L, c_{L+1}, \dots, c_H\} \\
 [P[a/b:[L, H]]] &= [P][f'] \quad f'(a_m) = b_m, \quad f'(\bar{a}_m) = \bar{b}_m \quad m \in [L, H] \\
 [\text{if } b \text{ then } P \text{ else } Q] &= \begin{cases} [P] & b \text{ true} \\ [Q] & b \text{ false} \end{cases} \\
 [K(e_1, \dots, e_h)] &= K_{n_1, \dots, n_h} \quad (e_i \downarrow n_i)
 \end{aligned}$$

$$K_{n_1, \dots, n_h} \stackrel{\text{def}}{=} [P\{^{n_1}/x_1, \dots, ^{n_h}/x_h\}] \quad \text{where } K(x_1:[L_1, H_1], \dots, x_h:[L_h, H_h]) \stackrel{\text{def}}{=} P \text{ and } n_i \in [L_i, H_i]$$

Each variable domain is restricted to a finite interval $[L, H]$, turning $\sum_{m \in \mathbb{N}}$ of the input rule into $\sum_{m=L}^H$.

Restriction and Renaming rules are also adapted with channel typing.

CCS-VP (Vccs.P)

```
open Interval

type Action =
  | Input of string * (string * Interval)
  | Output of string * AExp
  | Silent

type P =
  | ConstCall of string * AExp list
  | Act of Action * P
  | Sum of P * P
  | Parallel of P * P
  | Restrict of P * (string * Interval) list
  | Rename of P * (string * string * Interval) list
  | Nil
  | Conditional of BExp * P * P

type Vccs = string * (string * Interval) list * P
```

CCS (Pccs.P)

```
type channel = string

type Action =
  | Input of string
  | Output of string
  | Silent

type P =
  | ConstCall of string
  | Act of Action * P
  | Sum of P * P
  | Parallel of P * P
  | Restrict of P * string list
  | Rename of P * (Action * Action) list
  | Nil

type Pccs = string * P
```

Vccs also contains standard declaration for arithmetic and boolean expressions, that disappear, alongside Conditional, during compilation.

compile function

evaluateA

AExp $\rightarrow$ int

evaluateB

BExp $\rightarrow$ bool

compileP

Vccs $\rightarrow$ env $\rightarrow$ Pccs

getValuations

(string * Interval) list $\rightarrow$ (string * int) list list

compileDecl

Vccs $\rightarrow$ Pccs list

compile

Vccs list $\rightarrow$ Pccs list

```
let compile (vccs_list: Vccs list) : Pccs list =  
  
  let rec compileDecl (vccs_proc: Vccs) : Pccs list =  
  
    let rec compileP (Vccs -> env -> Pccs) =  
      let evaluateA = ...  
      let evaluateB = ...  
      :  
      // Core-function  
  
    match vccs_proc with  
    | name, [], body ->  
      [Pccs(name, compileP body [])]  
    | name, params, body ->  
      getValuations params  
      |> List.map (  
        fun env ->  
          Pccs(name + sfx env, compileP body env)  
        )  
  
  vccs_list |> List.collect compileDecl
```

getValuations function

```
let getValuations (parameters: (string * Interval) list) : (string * int) list list =  
  let domains = parameters |> List.map (fun (name, (lo, hi)) -> name, [lo .. hi])  
  let rec cartesianProduct domains =  
    match domains with  
    | [] -> [ [] ]  
    | (name, values) :: rest ->  
      [ for value in values do  
        for combo in cartesianProduct rest ->  
          (name, value) :: combo ]  
  cartesianProduct domains
```

- Computes the Cartesian product of all parameter domains
- **Example:** $K(x: [0,1], y: [0,2]) \mapsto K_{0_0} \dots K_{1_2}$
- `compileP body env` called once per valuation; the valuation seeds the initial `env`

compileP function

Input `Act(Input(ch, (x, [L,H])), P)`
Each $m \in [L, H]$ produces one branch compiled appending $(x \mapsto m)$ to env, after folded into a sum via `List.reduce`.

Conditional `Conditional(BExp, P, Q)`
Guard evaluated using `evaluateB` and used to tail call branch accordingly.

Rename `Rename(P, (from, to, [L,H]) list)`
Each `(fromCh, toCh, [L,H])` generates two pairs per m : $(a_m \rightarrow b_m)$ and $(\bar{a}_m \rightarrow \bar{b}_m)$, directly implementing f' .

Restrict `Restrict(P, (ch, [L,H]) list)`
Each `(ch, [L,H])` expands to $\{ch_L, \dots, ch_H\}$ via `List.collect`; the compiled process is restricted to the resulting flat name list.

ConstCall `ConstCall(K, AExp list)`

- No-argument case: name unchanged.
- General case: each argument evaluated and concatenated as a suffix `(K_1_2)`.

Sum, Parallel, Nil, Silent, Output: structural pass-throughs.

Program

Two execution modes:

File mode

```
../project-root $ dotnet run input.vccs input.vccs
Parsed successfully.
Compiled successfully.
Output written to: input.ccs
```

Interactive REPL

```
../project-root $ dotnet run
vccs> B = in(x:[0,1]).'out(x).B;
B = (in_0.'out_0.B + in_1.'out_1.B)
vccs> B = in(x:[0,1]).'out(x).B;
B = (in_0.'out_0.B + in_1.'out_1.B)
vccs> #list
B
vccs> #clear
Context cleared.
vccs> #quit
```

With undefined-reference warnings. And `--caal` compatible output format.

Examples

```
// Lecture Examples - Value-Passing CCS Test Suite
```

```
// --- Single-slot cell -----  
Cell = in(x:[0,3]).CellFull(x);  
CellFull(x:[0,3]) = 'out(x).Cell;
```

```
// --- Simple buffer -----  
Buf = recv(x:[0,3]).BufFull(x);  
BufFull(x:[0,3]) = 'send(x).Buf;
```

```
// --- 2-place FIFO buffer -----  
Fifo2 = in(x:[0,2]).Fifo1(x);  
Fifo1(x:[0,2]) = 'out(x).Fifo2 + in(y:[0,2]).Fifo0(x,y);  
Fifo0(x:[0,2], y:[0,2]) = 'out(x).Fifo1(y);
```

```
// --- 2-place unordered buffer -----  
Ubuf2 = in(x:[0,2]).Ubuf1(x);  
Ubuf1(x:[0,2]) = 'out(x).Ubuf2 + in(y:[0,2]).Ubuf0(x,y);  
Ubuf0(x:[0,2], y:[0,2]) = 'out(x).Ubuf1(y) + 'out(y).Ubuf1(x);
```

```
// --- Bounded counter -----  
Counter(x:[0,3]) =  
  'read(x).Counter(x)  
  + (if x < 3 then 'inc(x).Counter(x+1))  
  + (if x > 0 then 'dec(x).Counter(x-1));
```

```
// --- Binary semaphore -----  
Sem(s:[0,1]) =  
  (if s == 0 then acquire(a:[1,1]).Sem(1))  
  + (if s == 1 then release(r:[1,1]).Sem(0));
```

```
// --- Coffee machine -----
```

Results

```
Cell = (((in_0.CellFull_0 + in_1.CellFull_1) + in_2.CellFull_2) + in_3.CellFull_3);
CellFull_0 = 'out_0.Cell;
CellFull_1 = 'out_1.Cell;
CellFull_2 = 'out_2.Cell;
CellFull_3 = 'out_3.Cell;
Buf = (((recv_0.BufFull_0 + recv_1.BufFull_1) + recv_2.BufFull_2) + recv_3.BufFull_3);
BufFull_0 = 'send_0.Buf;
BufFull_1 = 'send_1.Buf;
BufFull_2 = 'send_2.Buf;
BufFull_3 = 'send_3.Buf;
Fifo2 = ((in_0.Fifo1_0 + in_1.Fifo1_1) + in_2.Fifo1_2);
Fifo1_0 = ('out_0.Fifo2 + ((in_0.Fifo0_0_0 + in_1.Fifo0_0_1) + in_2.Fifo0_0_2));
Fifo1_1 = ('out_1.Fifo2 + ((in_0.Fifo0_1_0 + in_1.Fifo0_1_1) + in_2.Fifo0_1_2));
Fifo1_2 = ('out_2.Fifo2 + ((in_0.Fifo0_2_0 + in_1.Fifo0_2_1) + in_2.Fifo0_2_2));
Fifo0_0_0 = 'out_0.Fifo1_0;
Fifo0_0_1 = 'out_0.Fifo1_1;
Fifo0_0_2 = 'out_0.Fifo1_2;
Fifo0_1_0 = 'out_1.Fifo1_0;
Fifo0_1_1 = 'out_1.Fifo1_1;
Fifo0_1_2 = 'out_1.Fifo1_2;
Fifo0_2_0 = 'out_2.Fifo1_0;
Fifo0_2_1 = 'out_2.Fifo1_1;
Fifo0_2_2 = 'out_2.Fifo1_2;
Ubuf2 = ((in_0.Ubuf1_0 + in_1.Ubuf1_1) + in_2.Ubuf1_2);
Ubuf1_0 = ('out_0.Ubuf2 + ((in_0.Ubuf0_0_0 + in_1.Ubuf0_0_1) + in_2.Ubuf0_0_2));
Ubuf1_1 = ('out_1.Ubuf2 + ((in_0.Ubuf0_1_0 + in_1.Ubuf0_1_1) + in_2.Ubuf0_1_2));
Ubuf1_2 = ('out_2.Ubuf2 + ((in_0.Ubuf0_2_0 + in_1.Ubuf0_2_1) + in_2.Ubuf0_2_2));
Ubuf0_0_0 = ('out_0.Ubuf1_0 + 'out_0.Ubuf1_0);
Ubuf0_0_1 = ('out_0.Ubuf1_1 + 'out_1.Ubuf1_0);
Ubuf0_0_2 = ('out_0.Ubuf1_2 + 'out_2.Ubuf1_0);
Ubuf0_1_0 = ('out_1.Ubuf1_0 + 'out_0.Ubuf1_1);
Ubuf0_1_1 = ('out_1.Ubuf1_1 + 'out_1.Ubuf1_1);
```

Limitations

① Interval types only

Enumeration types from the spec (e.g. $\{a, b, c\}$) are not supported.

② Domain explosion

n parameters each with domain size d generate d^n definitions. No dead-code elimination or deduplication.

③ No static type checking

Undefined constant references surface only as post-compilation warnings, not as errors before code generation.